

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Кафедра системного програмування і
спеціалізованих комп'ютерних систем

КУРСОВИЙ ПРОЄКТ

з дисципліни «Комп'ютерні мережі»
на тему: Маршрутизація в мережі передачі даних

Студента IV курсу, групи KB-22
за спеціальністю
123 «Комп'ютерна інженерія»
Крутогуз М. І.

Керівник: доцент, к.т.н., Мартинова О.

Національна оцінка _____

Кількість балів: ____ Оцінка: ECTS ____

Члени комісії _____ доцент, к.т.н. Мартинова О.

Київ 2025 рік

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

Кафедра системного програмування і
спеціалізованих комп'ютерних систем

Рівень вищої освіти — перший (бакалаврський)

Спеціальність 123 «Комп'ютерна інженерія»

**ЗАВДАННЯ
на курсовий проект студенту**

Крутогузу Максиму Ігоровичу

1. Тема проекту — Маршрутизація в мережі передачі даних керівник проекту доцент, к.т.н., Мартинова Оксана Петрівна.

2. Термін подання студентом проекту _____

3. Вихідні дані до проекту 2 регіональні мережі, кожна з яких має мінімум 12 комунікаційних вузлів, 2 канали — супутникові, середній ступінь мережі 3, ваги каналів 3, 5, 6, 8, 10, 12, 17, 20, 25, 27, алгоритм модифікована версія алгоритму Дейкстра.

4. Зміст пояснювальної записки Перелік термінів, умовних позначень та скорочень. Вступ. Теоретична частина. Опис розробленого програмного продукту. Дослідницька частина. Висновки. Додатки.

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1.	Вивчення літератури за тематикою проєкту	10.10.2025	
2.	Розроблення та узгодження технічного завдання	13.10.2025	
3.	Аналіз існуючих рішень	18.10.2025	
4.	Підготовка матеріалів першого розділу курсового проєкту	10.11.2025	
5.	Підготовка матеріалів другого розділу курсового проєкту	19.11.2025	
6.	Підготовка дослідницької частини курсового проєкту	25.11.2021	
7.	Оформлення документації курсового проєкту	30.11.2025 – 20.12.2025	
8.	Попередній огляд матеріалів курсового проєкту	05.12.2025 – 14.12.2025	
9.	Захист курсового проєкту	06.12.2025 – 14.12.2025	

Студент

Крутогуз

Максим, Крутогуз

Керівник проєкту

Оксана, Мартинова

АНОТАЦІЯ

Цей курсовий проект призначений для вивчення на практиці маршрутизації в мережі передачі даних. Навички гарного розуміння комп'ютерних мереж, зокрема як процес маршрутизації побудований із середини є ключовим для нашої спеціальності — 123 «Комп'ютерна інженерія». У даному проекті наведено теоретичні відомості про алгоритми маршрутизації — як модифікований алгоритм Дейкстри. І було розглянуто різницю від звичайного алгоритму Дейкстри та модифікованим, і роз'яснено його переваги над звичайним алгоритмом.

На основі цих алгоритмів було написано програму симуляції передачі пакетів в датаграмному режимі та віртуального каналу.

Перевагою даного проекту є те, що тут було використано модифікований алгоритм Дейкстри, що дає змогу визначити альтернативні шляхи, і це дозволяє краще розподіляти навантаження при передачі пакетів.

Ключові слова: комп'ютерні мережі, маршрутизація, модифікований алгоритм Дейкстри, датаграмний режим, віртуальний канал, C++, регіональні мережі, супутникові канали.

ABSTRACT

This course project is designed for practical study of routing in data transmission networks. Skills in understanding computer networks, particularly how the routing process is constructed, are key to our specialty—123 "Computer Engineering." This project provides theoretical information about routing algorithms, including a modified Dijkstra's algorithm. It discusses the differences between the standard Dijkstra's algorithm and the modified version, as well as explaining its advantages over the standard algorithm.

Based on these algorithms, a simulation program for packet transmission in datagram mode and virtual channel was developed. The advantage of this project is the use of the modified Dijkstra's algorithm, which allows for the determination of alternative paths, enabling better load distribution during packet transmission.

Keywords: computer networks, routing, modified Dijkstra's algorithm, datagram mode, virtual channel, C++, regional networks, satellite channels.

ТЕХНІЧНЕ ЗАВДАННЯ

ЗМІСТ

1. ВСТУП.....	2
2. ПРИЗНАЧЕННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ.....	2
3. СКЛАД ОБЧИСЛЮВАЛЬНОЇ СИСТЕМИ.....	2
4. ТЕХНІЧНІ ВИМОГИ.....	3
5. ВИМОГИ ДО АПАРАТНОГО ЗАБЕЗПЕЧЕННЯ.....	3
6. ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	3
7. ЕТАПИ РОЗРОБКИ.....	4

					ІАЛЦ.467400.001 ТЗ									
Зм.	Арк	№ докум	Підпис	Дата	Маршрутизація в мережі передачі даних. <i>Технічне завдання</i>				Літ.		Арк.	Арк-ів		
Розроб.	Крутогуз М. І.								К	-	-	1	4	
Перевір.	Мартинова О. П.								КПІ ім. Ігоря Сікорського, ФПМ гр. КВ-22					
Н. контр.														
Затв.	Мартинова О. П.													

1. ВСТУП

В даному курсовому проєкті потрібно розробити програму, яка моделює процес визначення маршруту передачі повідомлень в мережі передачі даних та заданої конфігурації та передачу повідомлень довільного розміру в режимах:

- віртуального каналу;
- дейтаграмному режимі.

2. ПРИЗНАЧЕННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Підставою для розробки є завдання на виконання курсового проєкту, затверджене кафедрою системного програмування і спеціалізованих комп'ютерних систем Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3. СКЛАД ОБЧИСЛЮВАЛЬНОЇ СИСТЕМИ

Розроблювальна система моделювання маршрутів у мережі повинна включати такі складові: конфігурація мережі згідно варіанту (див. на вхідні дані курсового проєкту); реалізація модифікованого алгоритму Дейкстри для пошуку маршрутів; провести серію тестів із використанням створеної мережі для визначення: час доставлення повідомлень з різною довжиною та різними типами їх передач.

					ІАЛЦ.467400.001 ТЗ	Аркуш 2
Зм.	Арк	№ докум	Підпи с	Дата		

4. ТЕХНІЧНІ ВИМОГИ

Вимоги до програмного продукту, що розробляється:

- Кількість регіональних мереж – 2
- Кожна із регіональних мереж містить мінімум – 12 комунікаційних вузлів.
- Кількість супутникових каналів – 2
- Алгоритм пошуку найкоротших маршрутів – модифікований алгоритм Дейкстри.

5. ВИМОГИ ДО АПАРАТНОГО ЗАБЕЗПЕЧЕННЯ

- Оперативна пам'ять 1Гб або більше
- Процесор Intel Core i3 або швидший

6. ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

- Сумісність із будь-якою із ОС: Windows 10/11, Linux, MacOS
- Зручний консольний інтерфейс для виконання необхідних дій
- Основна парадигма розроблюваною програми ООП

					ІАЛЦ.467400.001 ТЗ	Аркуш 3
Зм.	Арк	№ докум	Підпи с	Дата		

7. ЕТАПИ РОЗРОБКИ

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту
1.	Вивчення літератури за тематикою проєкту	10.10.2025
2.	Розроблення та узгодження технічного завдання	13.10.2025
3.	Аналіз існуючих рішень	18.10.2025
4.	Підготовка матеріалів першого розділу курсового проєкту	10.11.2025
5.	Підготовка матеріалів другого розділу курсового проєкту	19.11.2025
6.	Підготовка дослідницької частини курсового проєкту	25.11.2021
7.	Оформлення документації курсового проєкту	30.11.2025 – 20.12.2025
8.	Попередній огляд матеріалів курсового проєкту	05.12.2025 – 14.12.2025
9.	Захист курсового проєкту	06.12.2025 – 14.12.2025

Зм.	Арк	№ докум	Підпис	Дата

ІАЛЦ.467400.001 ТЗ

Аркуш 4

ЗМІСТ

ПЕРЕЛІК ТЕРМІНІВ, УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ.....	2
ВСТУП.....	3
РОЗДІЛ 1. ТЕОРЕТИЧНА ЧАСТИНА.....	4
1.1 Постановка задачі.....	4
1.2 Структура проектованої мережі.....	4
1.3 Опис та аналіз методів маршрутизації при передачі повідомлень.....	5
1.4 Модифікований алгоритм Дейкстри.....	5
1.5 Опис і аналіз запропонованого алгоритму при різних розмірах повідомлення та параметрах передачі.....	7
РОЗДІЛ 2. ОПИС РОЗРОБЛЕНОГО ПРОГРАМНОГО ПРОДУКТУ.....	8
2.1 Призначення та структура розробленого програмного продукту (ПП).....	8
2.2 Опис функцій та процедур, які реалізовано в програмі.....	9
2.2.1 Клас Controller.....	9
2.2.2 Класи Edge, Node, Graph та структура Weight.....	10
2.2.3 Клас PathAlgorithm.....	11
2.2.4 Клас Simulation і структура MessageStats.....	12
2.2.5 Класи Packet, Message та структура Transaction.....	14
2.2.6 Глобальні змінні, функції та клас Random.....	16
2.3 Результати функціонування розробленого ПП.....	17
РОЗДІЛ 3. ДОСЛІДНИЦЬКА ЧАСТИНА.....	22
3.1 Опис процесу тестування.....	22
3.2 Представлення результатів тестування, аналіз та порівняння отриманих результатів.....	23
3.2.1 Передача повідомлень різних розмірів при різних значеннях розміру пакетів та при різних параметрах передачі.....	23
3.2.2 Затримка між найбільш віддаленими робочими стнціями.....	25
3.2.3 Ймовірність доставлення повідомлення в залежності від розміру повідомлення.....	27
3.2.4 Оптимальний розмір пакету залежності від типу передавання та завантаженості мережі.....	28
ВИСНОВКИ.....	32
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ.....	34
ДОДАТОК А. КОНФІГУРАЦІЯ МЕРЕЖІ.....	35

					ІАЛЦ. 467200.004 ПЗ											
Зм	Ар	№ докум	Підпис	Дата												
.	к				<i>Маршрутизація в мережі передачі даних. Пояснювальна записка</i>					Літ.			Арк.		Арк-ів	
Розроб.		Крутогуз М. І.											1		45	
Прев.		Мартінова О. П.														
Н. контр.																
Затв.		Мартінова О. П.														
					КПІ ім. Ігоря Сікорського, ФПМ гр. КВ-22											

ПЕРЕЛІК ТЕРМІНІВ, УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ

ОС — операційна система

ООП — об'єктно-орієнтована парадигма

					ІАЛЦ. 467200.004 ПЗ	Аркуш 2
Зм.	Арк	№ докум	Підпи с	Дата		

ВСТУП

Даний курсовий проект призначення для поглиблення знань із дисципліни «Комп'ютерні мережі», та інших суміжних дисциплін, а саме: «Структури даних та алгоритми», «Теорія інформації та кодування», «Теорія ймовірностей», «Комп'ютерні системи», «Архітектура комп'ютерів». Виконання курсового проекту сприяє розширенню та поглибленню отриманих теоретичних знань щодо архітектури та структурної організації комп'ютерних мереж, принципів взаємодії робочих станцій, режимів передачі повідомлень в мережах передачі даних, формуванню вмінь аналізувати отримані результати, самостійного вирішення поставленої задачі оптимальним шляхом та оформлення документів курсового проекту відповідно до чинних правил і стандартів.

Для виконання цілей курсового проекту, передбачається розробка системи моделювання роботи комп'ютерної мережі, вивчення та реалізація алгоритмів визначення найкоротшого маршруту передачі повідомлень різних розмірів та аналіз службового та інформаційного трафіку в мережі при різних умовах передачі.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 3
Зм.	Арк	№ докум	Підпис	Дата		

РОЗДІЛ 1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Постановка задачі

Потрібно розробити програму, яка моделює комп'ютерну мережу заданої конфігурації. Також потрібно реалізувати можливість передавати повідомлень різної довжини за такими режимами: віртуального каналу, дейтаграмному режимі.

Розробка проекту складається із декількох етапів:

1. Конфігурація мережі передачі даних
2. Використання модифікованого алгоритму Дейкстра для пошуку найкоротших маршрутів між однієї робочої станцією мережі та всіма іншими.
3. Проведення серії тестів використовуючи побудовану модель мережі
4. Представлення отримані результати у вигляді графіків

1.2 Структура проектованої мережі

Проектована мережа буде складатися із двох регіональних мереж по мінімум 12 комунікаційних вузлів та двох супутників каналів. Спілкування між регіональними мережами буде ініціалізуватися завдяки супутників і декількох звичайних зв'язків між регіональними мережами. Середній ступінь мережі буде складати 3. Також в нашому моделювання будемо враховувати що для геостаціонарних супутників пропускна здатність передачі менше, а затримка та ймовірність виникнення помилок більше, ніж традиційним оптоволоконний способом передавання мережного трафіку.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 4
Зм.	Арк	№ докум	Підпи с	Дата		

Також в моєму варіанту наведені приклади ваг: 3, 5, 6, 8, 10, 12, 17, 20, 25, 27, які ми встановимо їх випадковим чином як ваги для зв'язків (зокрема для затримок між станціями) між робочими станціями маршрутизаторів.

1.3 Опис та аналіз методів маршрутизації при передачі повідомлень

При маршрутизації повідомлень виникає потреба в пошуку найшвидшого маршруту для передавання повідомлень. Наприклад, якщо повідомлення відправляється від певної робочої станції, то для передачі повідомлення до необхідного місця призначення повинен бути знайдений маршрут за яким повинні рухатися пакети із даними. Для вирішення цієї проблеми добре підходить теорія графів та алгоритми пошуку найшвидших шляхів. Наприклад, існує різноманітна кількість алгоритмів пошуку шляхів від простих (наприклад лавинний спосіб) до більш складаних і динамічних.

1.4 Модифікований алгоритм Дейкстри

Для пошуку найкоротшого маршруту будемо використовувати модифікований алгоритм Дейкстри. Для початку аналізу модифікованого методу, більш доречніше буде проаналізувати звичайний метод пошуку найкоротшого маршруту Дейкстри, а потім з'ясувати в чому різниця із модифікованим.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 5
Зм.	Арк	№ докум	Підпис	Дата		

Алгоритм пошуку найкоротшого шляху за алгоритмом Дейкстри:

1. Вибрати стартову вершину на графі і встановити значення відстані до нуля, а на інших вершинах встановити значення відстані до нескінченності.
2. Встановити та підтримувати список вже відвіданих вершин.
3. Вибрати вершину із найближчою відстані із стартової точки. Вона стає поточною точкою.
4. Для кожної вершини по сусідству, обрахувати відстань від поточної точки і порівняти її до попередньо записаної відстані. Якщо нова обрахована відстань менше, то оновити вершину.
5. Відколи всі сусіди перевірені, позначити поточну вершину як відвідана. Вона не буде перевірена знову.
6. Повернутися до кроку 3 і повторити процес доки всі вершини будуть відвідані.

Тепер повернемося до модифіковано алгоритму Дейкстри. Він використовує звичайний алгоритм Дейкстри, але після знаходження найкоротшого шляху він блокує ребра, які входять до найшвидшого маршруту. Після цього відбувається повторний прохід, що дозволяє знайти резервний шлях, якщо щось трапиться із основним або основний буде перевантажений.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 6
Зм.	Арк	№ докум	Підпис	Дата		

1.5 Опис і аналіз запропонованого алгоритму при різних розмірах повідомлення та параметрах передачі

Алгоритм не залежить від розміру повідомлення. Але при датаграмному режимі або віртуального каналу, робота алгоритму трішки відрізняється.

При передачі повідомлень в датаграмному режимі, то шлях обирається кожного разу коли пакет з'являється на маршрутизаторі. Тобто кожен маршрутизатор має свою таблицю маршрутів. Якщо згадати, що в реалізації модифікованого алгоритму Дейсктри є два типи маршрутів: один основний, а інший резервний. І для визначення маршруту в маршрутизаторі, виконується такий алгоритм: якщо ідентифікатор пакету парний, то 15%, що пакет буде використовувати резервний шлях; якщо ідентифікатор пакету непарний, то 7,5%, що пакет використовувати резервний шлях; інакше буде вибраний основний (найкоротший) шлях. Причина використання резервного шляху і певна ймовірність, що пакет буде направленим за ним полягає в тому, щоб розосередити трафік по різних маршрутах, аби вони менше заважали один одному.

При передачі повідомлень з використанням віртуального каналу полягає в тому, що для всіх повідомлень певного повідомлення резервується найкоротший шлях і всі пакети передаються за ним.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 7
Зм.	Арк	№ докум	Підпис	Дата		

РОЗДІЛ 2. ОПИС РОЗРОБЛЕНОГО ПРОГРАМНОГО ПРОДУКТУ

2.1 Призначення та структура розробленого програмного продукту (ПП)

Призначення розробленої програми для симуляції мережевого трафіку.

Тепер перейдемо до структури програми і для початку переглянемо файлову структуру проекту на рисунку 2.1.

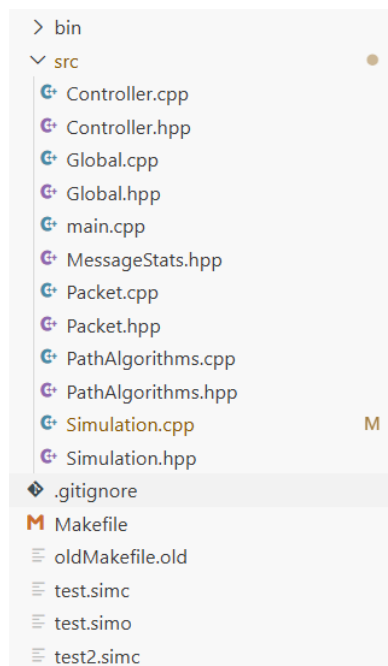


Рисунок 2.1 — Файлова структура проекту

Зверху можемо побачити директорію `bin`, в яку компілюється виконувальний файл. Трішки нижче можемо побачити, що проект містить 6 файлів з розширенням `hpp` та 6 файлів із розширенням `cpp` у директорії `src`. Ці файли реалізують всю логіку програми. Після вихідного коду проекту, є файл `.gitignore`, що дозволяє ігнорувати файли, які непотрібно завантажувати до репозиторію, далі доволі важливий файл — із назвою `Makefile`.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 8
Зм.	Арк	№ докум	Підпи с	Дата		

Потреба і користь від Makefile, це те що цей файл і технологія збірки, що використовує цей файл, дозволяє швидко і зручно за однією командою зібрати чи запустити проект, якого виконувальний файл збережеться в директорії bin. Також є файли із двома типами розширеннями simc та simo. Вони використовуються у симуляції розгляд їх буде пізніше.

2.2 Опис функцій та процедур, які реалізовано в програмі

В даному проекті є такі класи як: Controller, Edge, Node, Graph, Random, Packet, PathAlgorithm, Message і структури як: Weight, MessageStats, Transaction, Simulation. Також існують глобальні змінні: MTU, ROUTER_BUFFER_SIZE, header_size, TTL, messageStats та глобальні функції: generateRandomDouble.

Тепер спробуємо розглянути кожен із них і звернути на головні речі.

2.2.1 Клас Controller

Клас Controller призначений для роботи із користувачем, надаючи консольний інтерфейс для взаємодії. Детальний опис методів та даних можна побачити на рисунку 2.2.

При створенні цього класу вказується кількість супутників та не супутникових вузлів мережі, після цього можна запустити виконання контролера за допомогою команди run. При створенні контролера йде ініціалізація графу мережі (initGraph). Також цікавим є метод findPath, він перевіряє чи таблиця із основним маршрутом заповнена чи ні, якщо ні заповнює і далі повертає дані з якими зручно далі працювати.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 9
Зм.	Арк	№ докум	Підпис	Дата		

Інші методи вже призначені для того аби отримувати інформацію про доступні команди (метод help) або вивід інформації про поточний стан різних складових мережі (методи, які починаються на display), використовується у командах.

```
class Controller {      Maksym, 7 days ago • first commit but
public:
    Controller(int nonSatelliteNodeCount, int satelliteCount);
    void run();

    vector<int> findPath(int from, int to) const;

private:
    Graph _graph;

    void help() const;
    void initGraph();
    void displayGraphWeights() const;
    void displayFullGraphWeights() const;
    void displayNodeDistance(int id) const;
    void displayPath(int from, int to) const;
    void displayPathTable(int from) const;
    void displayBackDistance(int from, int to) const;
    void displayBackDistanceTable(int from) const;
    void displayErrorFrom(int from) const;
};
```

Рисунок 2.2 — Структура класу Controller

2.2.2 Класи Edge, Node, Graph та структура Weight

Ця структура та всі ці класи призначені для конструювання та конфігурування мережі, детальніше структура можна подивитися на рисунку 2.3.

Структура Weight використовується для полів із даними у класу Edge (дозволяє мати більш складні ваги, зокрема затримку та пропускну здатність каналу). Класи Edge та Node є складовими будівельними блоками для класу Graph. Тобто дані на основі цих класів зберігають інформацію, зокрема вузли

					ІАЛЦ. 467200.004 ПЗ	Аркуш 10
Зм.	Арк	№ докум	Підпи с	Дата		

мережі та матрицю суміжності (в цій матриці зберігаються зв'язки між іншими вузлами).

```
class Node {
public:
    bool isSatellite;
    vector<long long> distTable;
    vector<int> parentTable;
    queue<shared_ptr<Packet>> buffer;

    Node(bool isSatellite = false);

    void fillTable(const Graph& g, int id);
    vector<int> findReservedPath(int source, int target, const Graph& g);
    bool isCalculated() const;
};

class Graph {
public:
    int n;
    map<int, Node> nodes;
    vector<vector<Edge>> adj;
    vector<pair<int, int>> edgeEndpoints;

    Graph(int nonSatelliteNodeCount, int satelliteCount);
    void addNonDirectedEdge(int u, int v, const Weight& weight, ChannelType ct, double p_error);
    void output(ostream& stream) const;

    Node* getNode(int id);
    const Node* getNode(int id) const;
    Edge* findEdge(int from, int to);
    const Edge* findEdge(int from, int to) const;
};

struct Weight {
    double latency_ms;
    double bandwidth_mbps;

    Weight(double latency = 0.0, double bandwidth = 0.0);
    double calculate() const;
};

class Edge {
public:
    int to;
    int id;
    Weight weight;
    ChannelType type;
    double p_error;
    queue<shared_ptr<Packet>> buffer;

    Edge(int to, const Weight& w, ChannelType ct, double pe);

private:
    static int nextId;
};
```

Рисунок 2.3 — Класи Edge, Node, Graph та структура Weight

Що цікаво про клас Node, це те що в ньому є буфер в якому зберігається пакети (є обмеження в розмірі), також в цьому класі присутні таблиці і методи для пошуку найшвидшого маршруту і резервного маршруту.

Що цікавого про клас Edge, то в ньому є також буфер, але він працює трішки по іншому і в кожній сутності буферу є також час прибуття пакету до маршрутизатора і також в залежності чи з виділенням віртуального каналу чи в датаграмному режимі заповненість буферу для наступного пакету обраховується по різному.

2.2.3 Клас PathAlgorithm

Тепер розглянемо клас PathAlgorithm, це і є клас, який реалізує модифікований алгоритм Дейкстри, його можна розглянути на рисунку 2.3.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 11
Зм.	Арк	№ докum	Підпи с	Дата		

```

class PathAlgorithm
{
public:
    static pair<vector<long long>, vector<int>> dijkstra(const Graph& g, int s, const vector<bool> &disableEdge);
    static vector<int> reconstruct_nodes_from_parentEdges(int t, const vector<int> &parentEdge, const vector<pair<int, int>> &edgeEndpoints);
    static vector<int> reconstruct_edges_from_parentEdges(int target, const vector<int> &parentEdge, const vector<pair<int, int>> &edgeEndpoints);
};

```

Рисунок 2.3 — Клас PathAlgorithm

Він використовує статичні методи (тобто не створюються екземпляри цього класу). Перший метод `dijkstra` і реалізує сам алгоритм, але він використовує параметр `disableEdge` для того аби можна було після першого проходу знайти найкоротший шлях і для другого проходу цей шлях передати у параметри, аби можна було його зв'язки проігнорувати і найти резервний шлях. Всі інші методи є допоміжні і вони допомагають знайти шлях із попередньо заповнених таблиць.

2.2.4 Клас Simulation і структура MessageStats

Ці шаблони структур даних беруть активну участь у процесі симуляції пересилань повідомлень у програмі.

Розглянемо `MessageStats`, це структура, екземпляри якої знаходяться у глобальній множині, опис можна побачити на рисунку 2.4.

```

struct MessageStats {
    int packetsSent = 0;
    int bytesSent = 0;
    int bytesTotalSent = 0;
    int packetsReceived = 0;
    int bytesReceived = 0;
    int bytesTotalReceived = 0;
    int packetsDropped = 0;
    bool delivered = false;
    int firstSendTime = 0;
    int deliveryTime = 0;
};

```

Рисунок 2.4 — структура MessageStats

					ІАЛЦ. 467200.004 ПЗ	Аркуш 12
Зм.	Арк	№ докум	Підпи с	Дата		

Завдяки MessageStats, відстежується інформація про повідомлення у програмі і після виконання симуляції, аналізуючі дані, які використовують дану структуру допомагають отримати статистику.

Тепер повернімося до класу Simulation (детальніше рисунок 2.5), цей клас виконує весь процес симуляції. Він має доступ до графу (тобто до всієї мережі), має доступ до файлу (записує додатковий результат у файл), має час симуляції (мінімальна одиниця часу 1 мс), доступ до контролеру. Найбільш цікавим є пріоритетна черга із транзакцій, завдяки їй екземпляри класу можуть ініціалізувати пакет в будь-який час (як це робиться це буде пояснено пізніше про файл із розширенням simc) або дочекатися поки всі пакети будуть фрагментовані.

```
class Simulation {
public:
    Simulation(const string& inputfile, Graph& graph, Controller& controller);
    void run(const string& outputfile);

private:
    Graph& g;
    Controller& controller;
    priority_queue<Transaction> pq;
    long simulationTime;
    ofstream file;

    vector<Transaction> parseTransactionFile(const string& filepath);

    void processTransaction(Transaction& trans);
    void processBuffer();
    void processEdgeBuffers();

    void outputFileFields();
    void outputMessageStart(const Transaction& trans, const string& path);
    void outputPacketCreation(const shared_ptr<Packet>& packet, int nodeId, const string& path);
    void outputPacketTransmission(const shared_ptr<Packet>& packet, int fromNode, int toNode, const string& path);
    void outputPacketArrival(const shared_ptr<Packet>& packet, int node, const string& path);
    void outputPacketFail(const shared_ptr<Packet>& packet, int node, const string& path, string failReason);

    string getPath(const vector<int>& routeTable) const;
    int findNextHop(int currentNode, int destination, int seed);
    bool hasActivePackets();
    void updatePacketSentStats(const shared_ptr<Packet>& packet) const;
    void updatePacketReceiveStats(const shared_ptr<Packet>& packet) const;
    void updateDroppedStats(const shared_ptr<Packet>& packet) const;
    void outputStats() const;
};
```

Рисунок 2.5 — Клас Simulation

					ІАЛЦ. 467200.004 ПЗ	Аркуш 13
Зм.	Арк	№ докум	Підпи с	Дата		

Тепер ми зможемо поступово перейти до розгляду методів класу Simulation. Основний методом є метод run, що і дозволяє запустити симуляцію. Також цікавим методом є метод parseTransactionFile, який парсить simc файл (про цей файл згодом) і повертає список транзакції, які потрібно опрацювати. Методи, які розпочинаються із output виводять інформацію (зокрема щодо пакетів) у файл із розширенням simo (про нього також згодом). Також методи, які ми не можемо проігнорувати це processTransaction, processBuffer та processEdgeBuffers. Коротко про них, вони опрацьовують свій тип операцій, а саме: processTransaction — дозволяє обробити повідомлення (допомога у фрагментації), processBuffer — дозволяє опрацювати кожен маршрутизатор і перевірити чи він повинен щось робити в кожную одиницю симуляції, processEdgeBuffers — дозволяє опрацювати канали зв'язку (наприклад, якщо пакет повністю надійшов до маршрутизатора, то помістити в буфер маршрутизатора).

2.2.5 Класи Packet, Message та структура Transaction

Тепер перейдемо до розгляду класів Packet, Message та структури Transaction, детальніше можна переглянути на рисунку 2.6.

Почнемо наш розгляд із структури Transaction, ця структура є допоміжною для опрацювання повідомлень. Завдяки їй може ініціалізація початку передачі повідомлення в зазначений час t. Якщо маршрутизатора буфер не може опрацювати таку кількість пакетів повідомлення, то процес фрагментації може бути відкладений, змінивши параметр t. Також крім t є також додаткові технічні параметри: звідки і куди відправляти пакет після транзакції, розмір повідомлення і також є посилання на повідомлення.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 14
Зм.	Арк	№ докум	Підпис	Дата		

```

class Packet {
public:
    static Packet create(int src, int dst, int packetSize, int messageId, int type, const std::vector<int>* routeTable);
    static void reset();

    int getPacketId() const { return packetId; }
    int getMessageId() const { return messageId; }
    int getPacketSize() const { return packetSize; }
    int getType() const { return type; }
    long getTransmissionUntil() const { return transmissionUntil; }
    void setTransmissionUntil(long int newValue) { transmissionUntil = newValue; }
    int getSendingTime() const { return sendingTime; }
    void setSendingTime(int newValue) { sendingTime = newValue; }
    int getTTL() const { return ttl; }
    int getSrc() const { return src; }
    int getDst() const { return dst; }
    void decrementTTL() { ttl--; }
    const std::shared_ptr<const std::vector<int>& getRouteTable() const { return routeTable; }
    int getNextNode(const Graph& g, int currentNode) const;
    void incrementRoutePos() { routePos++; }
    void decrementRoutePos() { routePos--; }
    ~Packet();

private:
    Packet(int packetId, int src, int dst, int packetSize, int messageId, int type, const std::vector<int>* routeTable);
    static int nextPacketId;

    int packetId;
    int src;
    int dst;
    int packetSize;
    int messageId;
    int type;
    long int transmissionUntil;
    int sendingTime;
    int ttl;
    std::shared_ptr<const std::vector<int>> routeTable;

    int routePos;
};

class Message {
public:
    Message(int messageSize, int messageId, TransactionType type);

    int getDisplacement() const { return displacement; }
    void setDisplacement(int newValue) { displacement = newValue; }
    int getMessageSize() const { return messageSize; }
    int getMessageId() const { return messageId; }
    TransactionType getType() const { return type; }

    void setRoute(const vector<int>& route);
    const vector<int>& getRoute() const { return routeTable; }
    bool hasRoute() const { return !routeTable.empty(); }

private:
    int displacement;
    int messageSize;
    int messageId;
    TransactionType type;
    vector<int> routeTable;
};

struct Transaction {
    int t;
    int src;
    int dst;
    int size;
    TransactionType type;
    int id;
    shared_ptr<Message> message;

    Transaction(int t, int src, int dst, int size, TransactionType type, int id);
    bool operator<(const Transaction& other) const;
};

```

Рисунок 2.6 — Класи Packet, Message та структура Transaction

Тепер перейдемо до класу Message. Нащо варто звернути увагу, це на те що в повідомленні є зміщення (це ознака до якого байту було фрагментовано повідомлення), є маршрут (це лише актуально для виділення віртуального каналу) і різні технічні значення, наприклад ідентифікатор повідомлення.

І розглянемо останній клас в цьому пункті, це клас Packet. Це доволі важливий клас, оскільки його екземпляри переміщаються по мережі. Він містить багато геттерів та сеттерів для своїх полів. Розглянемо найважливіші поля: ttl – час життя пакету (зменшується після кожного маршрутизатора), routeTable – таблиця маршруту (актуально лише для режиму із виділенням віртуального каналу), src/dst — джерело та пункт призначення пакету, packetSize — розмір пакету, packetId — ідентифікатор пакету. Також окрему увагу варто звернути увагу на transmissionUntil та sendingTime, вони використовуються для службових потреб при передаванні по каналу зв'язку.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 15
Зм.	Арк	№ докum	Підпи с	Дата		

2.2.6 Глобальні змінні, функції та клас Random

В нас є такі глобальні змінні та функції як: MTU, TTL, ROUTER_BUFFER_SIZE, header_size, messageStats та generateRandomDouble.

Глобальні значення встановлюють по замовчуванню певні параметри: MTU – який максимальний розмір без фрагментації, TTL – який час життя пакету, ROUTER_BUFFER_SIZE – який розмір буферів маршрутизаторів, header_size – який розмір хедерів (службової інформації в пакеті). messageStats – це неупорядкована множина, в якій зберігається інформація для статистики, наприклад кількість відправлених, відкинути чи отриманих пакетів. Функція generateRandomDouble – повертає випадкове число між 0 та 1.

Тепер перейдемо до класу Random – це також спосіб отримувати випадкові числа між 0 та 1. Але насправді ми отримуємо псевдовипадкові числа — тобто на відмінну від generateRandomDouble, методи цього класу дозволяють отримувати постійні числа, які схожі на випадкові, що дуже добре для конфігурації мережі, наприклад затримки, ймовірність помилки на кожному каналі та інше. Повна структура класу Random на рисунку 2.7.

```
class Random {
public:
    Random(double initValue);
    double getValue();

private:
    double _currentValue;
};
```

Рисунок 2.7 — Клас Random

					ІАЛЦ. 467200.004 ПЗ	Аркуш 16
Зм.	Арк	№ докум	Підпис	Дата		

2.3 Результати функціонування розробленого ПП

Почнемо розглядати як ж сама програма функціонує, і для цього спершу розглянемо інтерфейс користувача (рисунок 2.8).

```
g++ -Wall -Wextra -std=c++17 -g bin/Controller.o bin/Global.o bin/main.o bin/Packet.o bin/PathAlgorithms.o bin/Simulation.o -o bin/App.exe
./bin/App.exe
Folainer$:
```

Рисунок 2.8 — Вигляд інтерфейсу користувачі відразу після запуску програми

Щоб подивитися команди потрібно написати `help` або скорочено `h`, і після ведення команди допомоги, можна побачити всі доступні команди у програмі (рисунок 2.8). Команди, які містять в собі назви як `mtu`, `buffer_size`, `ttl`, не будемо розглядати, оскільки вони дозволяють подивитися або змінити значення глобальних змінних, які вже було розглянуто.

```
Information about available commands:
h|help                Shows information about available commands
show graph            Shows information about graph
show weight           Shows information about weights of graph
show full_weight      Shows information about full weights of graph: latency_ms and bandwidth_mbps
show mtu              Shows global mtu in the network
show buffer_size      Shows global router buffer size in the network
show ttl              Shows global ttl for new packet in the network
show distance <id>    Shows information about dijkstra's short path
show path <from> <to> Shows the shortest path using dijkstra's algorithm
show path_table <from> Shows the shortest path table using dijkstra's algorithm in a specific router
show backup_path <from> <to> Shows the backup path
show backup_path_table <from> Shows the backup path table using modified dijkstra's algorithm in a specific router
show error_from <from> Shows errors in edges from selected node
change mtu <value>     Changes global mtu in the network
change buffer_size <value> Changes global router buffer size in the network
change ttl             Changes global ttl for new packet in the network
simulate <infile> <ofile> Simulates network packets transmission using input and output files
e|exit                Exits
```

Рисунок 2.8 — Доступні команди у програмі

Інші доступні команди:

- `show graph` – відображає граф у матриці суміжності
- `show weight` – для кожного каналу зв'язку показує обраховану вагу

					ІАЛЦ. 467200.004 ПЗ	Аркуш 17
Зм.	Арк	№ докум	Підпи с	Дата		

- show full_weight – працює аналогічно, як і show weight, але не обраховує остаточну вагу, тобто виводить затримку та пропускну здатність кожного каналу
- show distance <id> – показує відстані від заданого вузла до всіх інших
- show path <from> <to> – виводить найкоротший маршрут між заданими вузлами
- show path_table <from> – працює аналогічно до верхнього методу, але виводить із заданого вузла найкоротші маршрути до всіх вузлів мережі
- show backup_path <from> <to> – виводить резервний маршрут між заданими вузлами
- show backup_path_table <from> – працює аналогічно до верхнього методу, але виводить із заданого вузла резервні маршрути до всіх вузлів мережі
- show error_from <from> – показує всі канали зв'язку від заданого вузла і показує для кожного каналу зв'язку ймовірність виникнення помилок та тип каналу (дуплекс чи напівдуплекс)
- e|exit – дає можливість завершити програму
- simulate — являється важливою командою, оскільки вона із файлу із заданим синтаксисом (файл із розширенням simc), виводить всі операції у інший файл (файл із розширенням simo), також статистичні дані ця команда виводить у консоль. Роботу цієї команди розглянемо згодом

Розглянемо детальне про симуляцію та файли із розширенням simc та simo. Ці два розширення використовуються в команді simulate і передаються як параметри: один вхідний файл, а інший вихідний.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 18
Зм.	Арк	№ докум	Підпис	Дата		

Перед розглядом процесу симуляції та файлу із розширенням simo, розглянемо файл із розширенням simc, оскільки саме він програмує передавання повідомлень по мережі, приклад формату файлу можна побачити на рисунку 2.9.

```
test2.simc
1  0: 0->1 3000 =
2  1: 21->8 1500 =
3  2: 10->12 2000
4  4: 22->8 2400 =
```

Рисунок 2.9 — Приклад файлу з розширенням simc

Тепер перейдемо до форми запису, який можна побачити на рисунку 2.10. Першими даними — це є час, який записується спочатку перед двокрапкою, далі йде джерело та місце призначення, які пишуться поруч відокремлюючись стрілкою. Після цього, записується розмір повідомлення, яке хочемо передати. І в кінці може стояти знак дорівнює або ні: якщо поставити, то будемо мати справу із виділенням віртуального каналу, якщо ні, то будемо мати справу із датаграмним режимом.

джерело
час призначення
розмір

0: 0->1 3000 =

Рисунок 2.10 — Форма запису у файл simc

Після того як розібралися із способом як ми можемо спланувати передачу повідомлень, то далі будемо розбирати результати роботи симуляції. Тому нагадаю, під час симуляції ми надаємо два файли один файл із розширенням simc (самі заповнюємо), а інший файл із розширенням simo

					ІАЛЦ. 467200.004 ПЗ	Аркуш 19
Зм.	Арк	№ докум	Підпис	Дата		

(результат симуляцій), а також як бонус отримуємо статистику в консоль, тому нумо розбиратися в цьому.

Тоді почнемо із файлу із розширенням `simo`. Приклад файлу можна побачити на рисунку 2.12, який генерується на основі файлу рисунок 2.11. На рисунку можемо побачити: тип операції, час симуляції при виконанні операції, ідентифікатор повідомлення, ідентифікатор пакету (в деяких операціях відсутній), розмір повідомлення/пакету. Далі може бути по різному, наприклад у фігурних дужках для операцій з пакетів показується в якому маршрутизаторі (якщо одне число) чи каналу зв'язку (якщо два числа відокремлені стрілкою) відбувається операція. В квадратних дужках позначено джерело та пункт призначення. Для віртуального каналу з права вкінці записаний повний маршрут пересування у мережі.

```
0: 0->5 2400
0: 5->7 2000 =
```

Рисунок 2.11 — Файл із розширенням `simc` (генерує `simo` нижче)

type	time	msg_id	pck_id	size
MCD	0	1	-	2400 [0->5]
PCD	0	1	1	1500 {0}[0->5]
PCD	0	1	2	1000 {0}[0->5]
MCV	0	2	-	2000 [5->7] 5->6->7
PCV	0	2	3	1500 {5}[5->7] 5->6->7
PCV	0	2	4	600 {5}[5->7] 5->6->7
PTD	1	1	1	1500 {0->6}[0->5]
PTV	1	2	3	1500 {5->6}[5->7] 5->6->7
PTD	7	1	2	1000 {0->11}[0->5]
PTV	13	2	4	600 {5->6}[5->7] 5->6->7
PTV	13	2	3	1500 {6->7}[5->7] 5->6->7
PTD	24	1	1	1500 {6->5}[0->5]
PTD	26	1	2	1000 {11->12}[0->5]
PRD	36	1	1	1500 {5}[0->5]
PTV	40	2	4	600 {6->7}[5->7] 5->6->7
PRV	40	2	3	1500 {7}[5->7] 5->6->7
PTD	53	1	2	1000 {12->3}[0->5]
PTD	65	1	2	1000 {3->5}[0->5]
PRV	66	2	4	600 {7}[5->7] 5->6->7
PRD	73	1	2	1000 {5}[0->5]

Рисунок 2.12 — Файл із розширенням `simo`

					ІАЛЦ. 467200.004 ПЗ	Аркуш 20
Зм.	Арк	№ докум	Підпи с	Дата		

Поясню детальніше про тип операції:

- Перший символ може бути М або Р, тобто це означає чи це пакет чи повідомлення приймають роль в операції
- Другий символ може бути С, Т, R та F. С — це означає створення, Т – це означає передача, R – означає отримання в пункті призначення. Щодо F – то на прикладі не має, але воно використовується в помилкових ситуаціях, а саме: час життя пакета вийшов, буфер переповнений маршрутизатора до якого було надісланий пакет, або помилка з’явилася при передачі по каналу зв’язку
- Третій символ може бути D або V – це означає чи операція для датаграмного режиму чи із виділенням віртуального каналу

Перейдемо до пояснення щодо консольного виводу і приклад виводу можна побачити на рисунку 2.13. Тут можна побачити статистичну інформацію щодо всіх повідомлень і на рисунку лише для повідомлення із ідентифікатором 1. Наприклад, скільки пакетів відправлено (з хедерами та без), отримано (з хедерами та без), чи повідомлення доставлено повністю, скільки відкинуто пакетів чи за який час повідомлення було відправлено.

```
=====
Message ID: 1
-----
Packets sent:      2
Bytes sent:        2400
Total bytes sent:  2500
Packets received:  2
Bytes received:    2400
Total bytes received: 2500
Packets dropped:   0
Delivered:         YES
Delivered percentage: 100%
First send time:   0
Delivery time:     73
=====
```

Рисунок 2.13 — Приклад виведення статистичної інформації

					ІАЛЦ. 467200.004 ПЗ	Аркуш 21
Зм.	Арк	№ докум	Підпис	Дата		

РОЗДІЛ 3. ДОСЛІДНИЦЬКА ЧАСТИНА

3.1 Опис процесу тестування

Перейдемо до опису процесу тестування, але спочатку розглянемо структуру розробленої мережі (рисунок 3.1).

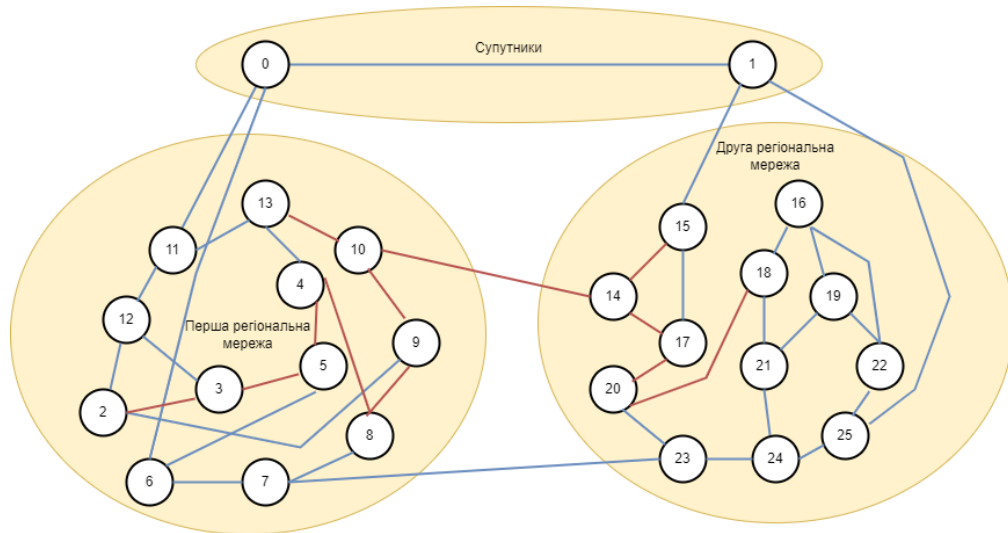


Рисунок 3.1 — Конфігурація мережі

Для тестування було застосовано засоби електронних таблиць (рисунок 3.3) та файли із розширенням `simc` (рисунок 3.2). Додаткова статистика отримувалася із консолі. Варто звернути увагу, що було проведено декілька спроб при тестуванні для отримання середнього значення, яке краще для порівняння системи в цілому.

```
0: 3->21 256000 =
0: 0->7 256000
0: 1->24 256000
0: 2->22 256000
0: 19->12 256000
```

Рисунок 3.2 — Файл із розширенням `simc` для тестування

					ІАЛЦ. 467200.004 ПЗ	Аркуш 22
Зм.	Арк	№ докум	Підпис	Дата		

3.2 Представлення результатів тестування, аналіз та порівняння отриманих результатів

3.2.1 Передача повідомлень різних розмірів при різних значеннях розміру пакетів та при різних параметрах передачі

Для цього пункту, було складено два графіка (рисунок 3.5 та 3.6) перший із даних рисунку 3.3 а інший із даних рисунку 3.4.

	Розмір	1000	4000	16000	64000	256000
	Номер спроби					
датаграмного режиму	1	119	166	412	1018	4271
	2	109	215	370	1163	4128
	3	65	209	410	1289	4122
	4	109	201	358	1169	4045
	5	109	173	382	1317	4188
	6	109	188	332	1080	4114
	7	125	175	402	1071	4127
	8	109	151	368	1160	4276
	Середня	106.75	184.75	379.25	1158.375	4158.875
віртуального каналу	Номер спроби					
	1	109	163	405	1,296.00	4861
	2	109	163	405	1,296.00	4861
	3	109	163	405	1,296.00	4861
	4	109	163	405	1,296.00	4861
	5	109	163	405	1,296.00	4861
	6	109	163	405	1,296.00	4861
	7	109	163	405	1,296.00	4861
	Середня	109	163	405	1,296.00	4861

Рисунок 3.3 — Файл для роботи із електронними таблицями (без додаткового трафіка)

	Розмір	1000	4000	16000	64000	256000
	Номер спроби					
Для датаграмного режиму	1	110	222	590	1911	7445
	2	143	262	486	1917	7652
	3	152	205	580	2080	7488
	4	109	312	583	1788	7450
	5	135	200	663	1792	7524
	6	221	271	499	1950	7372
	7	137	210	549	2023	7394
	8	137	201	553	2088	7159
	Середня	143	235.375	562.875	1943.625	7435.5
Для віртуального каналу	Номер спроби					
	1	135	218	594	2,081.00	7918
	2	135	183	593	1,946.00	7653
	3	109	179	564	2,055.00	7683
	4	136	187	619	2,082.00	7520
	5	135	190	567	2,053.00	7677
	6	160	179	650	1,893.00	7957
	7	133	181	583	2,164.00	7759
	Середня	134.75	190.125	592.125	2,041.38	7,737.75

Рисунок 3.4 — Файл для роботи із електронними таблицями (з додатковим трафіком)

					ІАЛЦ. 467200.004 ПЗ	Аркуш 23
Зм.	Арк	№ докум	Підпис	Дата		



Рисунок 3.5 — Взаємозв'язок між кількістю переданої інформації до часу (без додаткового трафіку)



Рисунок 3.5 — Взаємозв'язок між кількістю переданої інформації до часу (з додатковим трафіком)

Зм.	Арк	№ докум	Підпис	Дата

ІАЛЦ. 467200.004 ПЗ

Аркуш 24

Бачимо на рисунку 3.5, що за віртуальним каналом час передачі значно більший ніж для датаграмного режиму (без додаткового трафіку). Хоча для менших значень переданої інформації, зокрема для 4000 байт, можна побачити, що віртуальним режим трішки швидший (він використовує найшвидший маршрут). Тому можемо зробити висновки, що при маленький розмірах передача повідомлень дуже незначно більше за датаграмним режимом, і причина цього це те, що передача датаграми передаються за різними маршрутами і є висока ймовірність, що маршрут буде довший. Щодо більший розмірів тут картина інша і різниця більш суттєва і причина сповільнення віртуального каналу, може пояснюватися, що при передачах повідомлень потрібен додатковий шлях маршрутизаторам для обробки пакетів, а якщо пакету будуть проходити різні маршрути (датаграмний режим), то за той самий час різні маршрутизатори будуть опрацьовувати пакети паралельним чином.

Також глянемо на рисунок 3.6, різниця між попереднім це те, що крім основного повідомлення є ще додатковий трафік, який може проходити крізь ті самі маршрутизатори, і можемо зробити висновки, що це трафік перешкоджає іншим резервним шляхам датаграм, оскільки резервні шляхи трішки довше і їх користь зменшує завантаженість маршрутизаторів із обробки пакетів іншого трафіку.

3.2.2 Затримка між найбільш віддаленими робочими стнціями

Було проведені розрахунки (рисунок 3.6) між віддаленими маршрутизаторами при різних типах передачі та отриманий графік (рисунок 3.7). Розмір повідомлення лише 1000 байт (лише один пакет).

					ІАЛЦ. 467200.004 ПЗ	Аркуш 25
Зм.	Арк	№ докум	Підпи с	Дата		

	Спроби	1	2	3	4	5	6	7	8	Загалом
Датаграмний	2->16	91	91	91	91	133	107	91	101	99.5
	12->25	155	165	111	127	111	155	155	165	143
	5->21	101	144	101	101	101	101	101	114	108
	4->19	87	130	87	87	87	98	157	87	102.5
	11->22	130	120	120	120	150	120	120	149	128.625
Віртуальний	3->18	84	93	93	93	93	93	93	166	101
	2->16	91	91	91	91	91	91	91	91	91
	12->25	111	111	111	111	111	111	111	111	111
	5->21	101	101	101	101	101	101	101	101	101
	4->19	87	87	87	87	87	87	87	87	87
	11->22	120	120	120	120	120	120	120	120	120
	3->18	93	93	93	93	93	93	93	93	93

Рисунок 3.6 — Дані щодо вимірювань між віддаленими маршрутизаторами

Тестування затримок між найвідалішими вузлами мереж

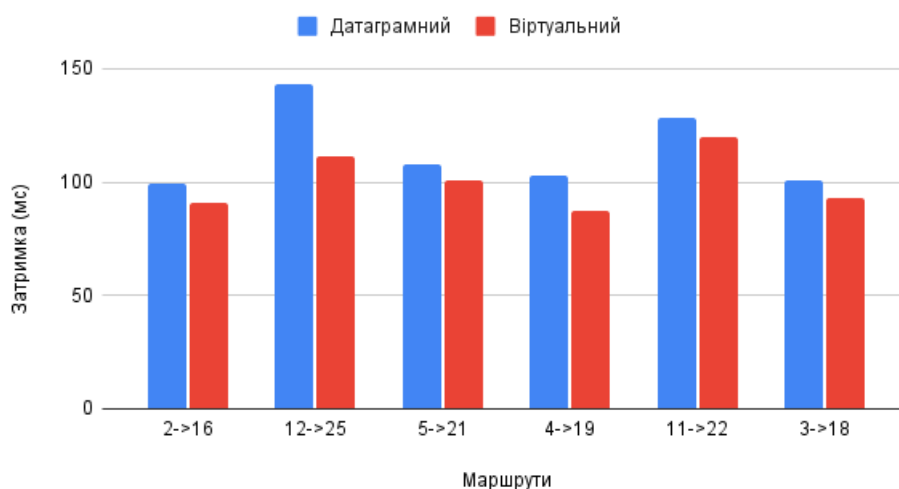


Рисунок 3.7 — Діаграма щодо затримок між різними маршрутизаторами

Тепер можемо зробити висновки, що найдовші затримки при передачі пакета може бути близько 150 мс. До того ж ми бачимо, що віртуальний канал швидше передає пакет. Причина слугує в тому: по-перше не має перешкоджаючого трафіку, по друге у віртуальному каналі використовується завжди найшвидший маршрут і оскільки передається один пакет не має затримок при обробленні маршрутизаторів.

Зм.	Арк	№ докум	Підпи с	Дата

ІАЛЦ. 467200.004 ПЗ

Аркуш 26

3.2.3 Ймовірність доставлення повідомлення в залежності від розміру повідомлення

Тепер зробимо тест щодо чи все повідомлення буде доставлено до пункту призначення чи буде відкинуто. Для цього формуємо дані (рисунок 3.8) та діаграму (рисунок 3.9). Дані для кожної спроби формувалися за допомогою 5 тестів, тобто загалом було 50 тестів.

Спроби		1	2	3	4	5	6	7	8	9	10	
Датаграмний	Розмір повідомлення											
	50,000	0.6	1	1	0.6	1	0.8	1	0.8	0.8	1	86.00%
	100,000	0.6	0.2	1	0.8	0.8	1	0.8	0.8	0.6	1	76.00%
	200,000	0.4	0.4	0.4	0.4	0.4	0.2	0	0.6	0.6	0.2	36.00%
	400,000	0	0.2	0	0	0	0.2	0.2	0	0	0	6.00%
Віртуальний	800,000	0	0	0	0	0	0	0	0	0	0	0.00%
	50,000	1	0.8	0.8	0.8	0.8	0.6	0.8	1	1	1	86.00%
	100,000	0.6	0.4	0.6	0.6	0.8	0.4	1	0.8	0.8	0.8	68.00%
	200,000	0.8	0.4	0.8	0.2	0.8	0.6	0.6	0.4	1	0.6	62.00%
	400,000	0	0	0	0	0	0	0	0	0	0	0.00%
	800,000	0	0	0	0	0	0	0	0	0	0	0.00%

Рисунок 3.8 — Дані щодо успішності передавання повідомлення в залежності його розміру

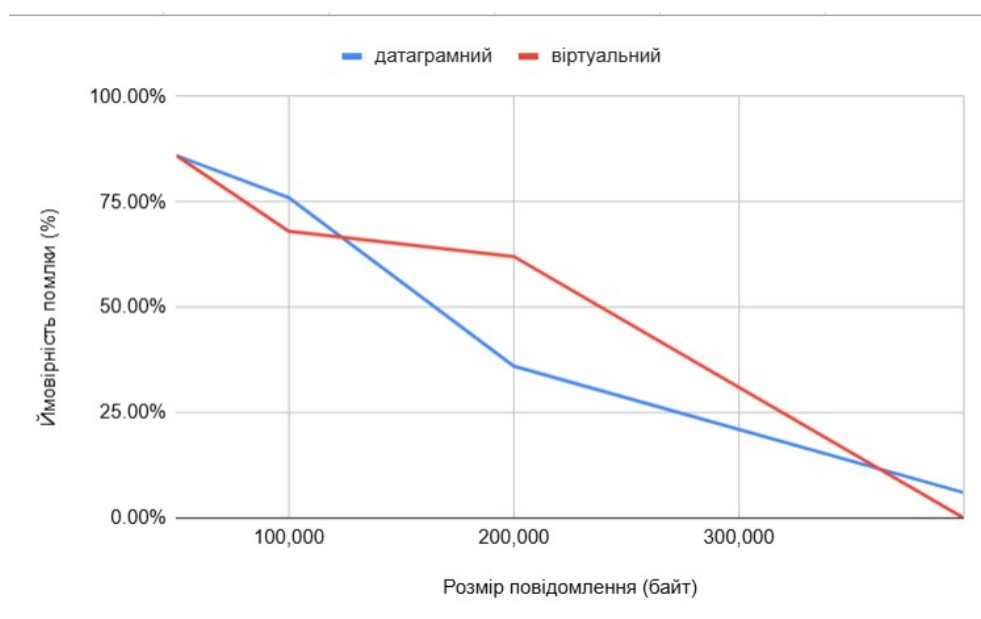


Рисунок 3.9 — Діаграма, що показує успішність передавання повідомлення в залежності від його розміру

Зм.	Арк	№ докум	Підпис	Дата

ІАЛЦ. 467200.004 ПЗ

Аркуш 27

Тепер перейдемо до його аналізу, бачимо постійні коливання в залежності від тип передавання інформації. Але хотів зазначити дані при розмірі повідомлення 800,000 байтів не були включені до діаграми. Також ми бачимо спад за віртуальним каналом, що прямує до нуля. Причина виникнення нульового відсотку (при передавання 800,000 байтів та при передавання 400,000 у віртуальному каналі) передавання даних полягає в тому, що виникає інша проблема, а саме: переповнення буферів маршрутизаторів при дуже інтенсивному передаванні великих повідомлень (рисунок 3.10). Тому графік вкінці трішки викривлений. Щодо коливання на початку, звичайно можна робити теорії, але на мою думку правильний висновок важко зробити, оскільки найкоротший шлях (використовується у віртуальному каналі) не означає, що він найбільш безпечніший для передавання повідомлень. Тому тестування за одним маршрутом не дозволить побачити повну та чітку картину ситуації.

```
PFV 282 1 274 1500 {5}[3->18] 3->5->6->7->23->20->18
Failed reason: Packet dropped (buffer full)
PRV 282 1 4 1500 {18}[3->18] 3->5->6->7->23->20->18
PFV 283 1 275 1500 {5}[3->18] 3->5->6->7->23->20->18
Failed reason: Packet dropped (buffer full) You, 20 minutes ago
PFV 284 1 276 1300 {5}[3->18] 3->5->6->7->23->20->18
Failed reason: Packet dropped (buffer full)
```

Рисунок 3.10 — Отримувана помилка

3.2.4 Оптимальний розмір пакету залежності від типу передавання та завантаженості мережі

Почнемо тестування для визначення найоптимальнішого розміру пакету при різних режимах передавання. Отримуємо дві таблиці із даними (рисунки 3.11 та 3.12) та два графіки (рисунки 3.13 та 3.14).

					ІАЛЦ. 467200.004 ПЗ	Аркуш 28
Зм.	Арк	№ докум	Підпи с	Дата		

	ROUTER_BUFFER_SIZE	MTU	Спроби (Час)										Разом
			1	2	3	4	5	6	7	8	9	10	
Датаграмний	500	750	3326	3196	3291	3331	3195	3269	2948	3271	3351	3280	3246
	250	1500	1818	1655	1606	1679	1635	1783	1549	1765	1829	1605	1692
	125	3000	955	951	899	939	887	937	853	934	1084	957	940
	63	6000	701	598	800	684	642	636	670	679	634	641	669
	31	12000	730	592	585	625	668	720	727	506	621	720	649
Разом													
Віртуальний	500	750	3796	3796	3796	3796	3796	3796	3796	3796	3796	3796	3796
	250	1500	1947	1947	1947	1947	1947	1947	1947	1947	1947	1947	1947
	125	3000	1082	1082	1082	1082	1082	1082	1082	1082	1082	1082	1082
	63	6000	679	679	679	679	679	679	679	679	679	679	679
	31	12000	513	513	513	513	513	513	513	513	513	513	513

Рисунок 3.11 — Тестування щодо розмір пакету та розміру буфера маршрутизатора без додаткового трафіку

	ROUTER_BUFFER_SIZE	MTU	Спроби (Час)										Разом
			1	2	3	4	5	6	7	8	9	10	
Датаграмний	500	750	4900	4756	4709	4648	4852	4657	4779	4663	4707	4744	4742
	250	1500	2473	2288	2581	2316	2417	2450	2668	2440	2465	2513	2461
	125	3000	1346	1404	1399	1355	1485	1434	1351	1503	1459	1455	1419
	63	6000	883	996	979	968	859	902	1072	969	961	912	950
	31	12000	773	699	799	867	759	763	779	738	674	877	773
Разом													
Віртуальний	500	750	5266	5201	5191	5119	4913	5116	5286	5064	5200	5449	5181
	250	1500	2627	2730	2557	2703	2886	2645	2548	2825	2741	2842	2710
	125	3000	1545	1573	1404	1515	1435	1483	1463	1493	1463	1554	1493
	63	6000	946	849	912	903	992	937	911	949	957	932	929
	31	12000	729	709	683	710	675	619	742	613	620	728	683

Рисунок 3.12 — Тестування щодо розмір пакету та розміру буфера маршрутизатора з додатковим трафіком

Тепер перейдемо до даних і що ми бажали отримати. Якщо брати маршрутизатори з однакової кількістю пам'яті, то постає задача обиравання найоптимальнішого розміру пакету, але якщо збільшувати його розмір то розмір буфера маршрутизатора також зменшення, це і видно на вимірюваннях. Також було проведено два вимірювання без додаткового трафіку у мережі і без додаткового трафіку і кожен із цих вимірювань містить заміри для віртуального каналу та датаграмного режиму.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 29
Зм.	Арк	№ докум	Підпи с	Дата		

Перейдемо до аналізу отриманих даних і спочатку почнемо аналіз із графіку (рисунок 3.13), який не містить додаткового трафіку під час вимірювань. Ми бачимо що чим більший розмір пакету тим краще для будь-якого режиму: для віртуального каналу та датаграмного режиму. Але питання чому таке трапляється і чому в реальних застосувань використовуються не дуже великий розмір для пакету, причина полягає в двох речах: перше в реалізації моєї програми оброблення пакетів маршрутизаторів завжди 1 мс і не змінюється від розміру пакету, друге під час тестування було зроблені виміри під час передачі одного великого повідомлення і через те не було випадків де пакет не набував максимально можливого розміру (лише останній пакет міг би обрізаним). Тому через ці фактори ми не можемо обрати дуже велике значення для розміру пакету. Причини щодо того що при малих розмірів пакету віртуальних канал переважав над датаграмним, а потім інакше — у віртуальному каналі всі пакету йдуть за тим самим маршрутом тим самим перевантажуючи канали зв'язку оскільки вони обмежені в кількості можливої передачі даних.

Тепер глянемо на інший графік (рисунок 3.14) тут схожа картина із попереднім графіком, але різниця між датаграмним режимом та віртуальним каналом не така велика і причина цього є те що пакети інших повідомлень (сторонніх повідомлень) використовують в якісь мірі частину пропускну здатності мережі і тим самим приріст швидкості від датаграмного режиму при великих обсягах передачі даних не спостерігається.

					ІАЛЦ. 467200.004 ПЗ	Аркуш 30
Зм.	Арк	№ докум	Підпис	Дата		

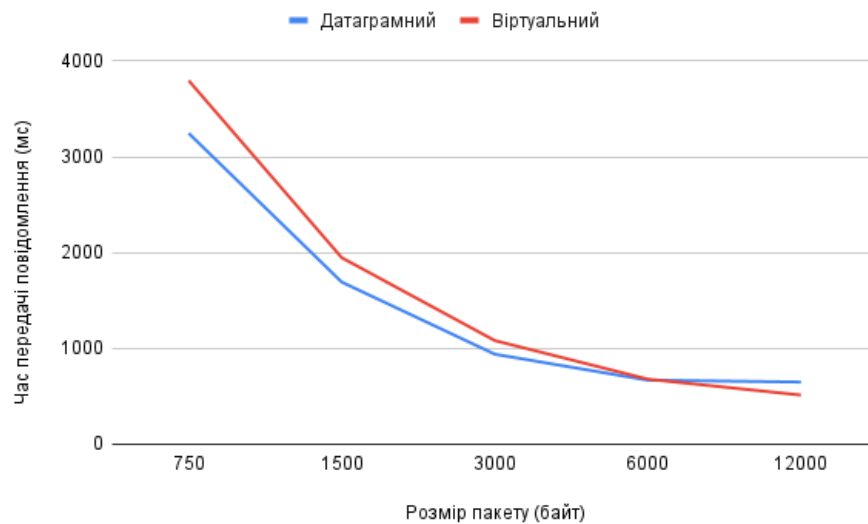


Рисунок 3.13 — Графік тестування щодо розміру пакету та розміру буфера маршрутизатора з додатковим трафіком

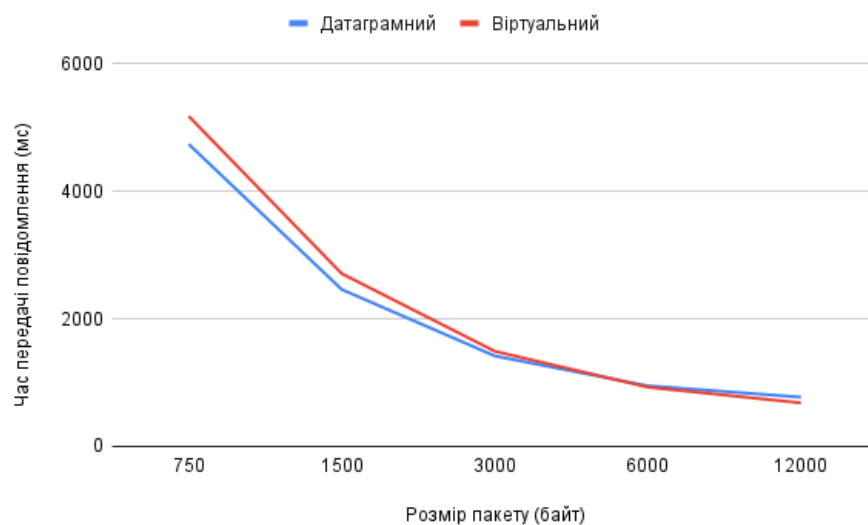


Рисунок 3.14 — Графік тестування щодо розміру пакету та розміру буфера маршрутизатора без додатковим трафіком

Зм.	Арк	№ докум	Підпис	Дата

ВИСНОВКИ

Під час проекту проекту було успішно розроблено програмний продукт написаний мовою C++, який симулює роботу комп'ютерної мережі складної конфігурації: два регіональні мережі та два супутника. Програма може реалізувати два способи передачі інформації: датаграмний режим та режим віртуального каналу.

Щодо алгоритмічної складової програми він використовує модифікований алгоритм Дейкстри. Його відмінність від класичного алгоритму полягає не лише в знаходженні основного (найкоротшого) шляху, а також знаходження резервного шляху, шляхом блокування ребер основного маршруту. Цей підхід дозволяє краще розподіляти навантаження: у датаграмному режимі частина пакетів направляється резервним шляхом для розвантаження мережі.

Перейдемо до результатів дослідження мережі у програмі. По-перше, датаграмний режим при передачі великих повідомлень виявився швидшим за режим віртуального каналу. Це пояснюється тим, що дозволяє різним маршрутизаторам обробляти їх паралельно, тоді як у віртуальному каналі все навантаження лягає на один ланцюжок вузлів. По-друге, для невеликих повідомлень або поодиноких пакетів режим віртуального каналу демонструє меншу затримку та працює трішки швидше. Це пов'язано з використанням гарантовано найкоротшого маршруту та відсутністю потреби у складній маршрутизації кожного окремого пакету після встановлення з'єднання. Максимальна зафіксована затримка склала близько 150 мс. По третє, при інтенсивній передачі дуже великих обсягів даних (наприклад, 800 000 байт)

					ІАЛЦ. 467200.004 ПЗ	Аркуш 32
Зм.	Арк	№ докум	Підпис	Дата		

режим віртуального каналу показав схильність до збоїв (успішність доставки падала до 0%). Це трапляється через переповнення буферів маршрутизаторів, оскільки весь потік даних йде через одні й ті самі вузли, не маючи можливості перерозподілитися. По четверте, наявність додаткового трафіку в мережі зменшує перевагу дейтаграмного режиму, оскільки резервні шляхи стають більш завантаженими, що нівелює користь від розпаралелювання.

Тепер перейдемо до оптимального розміру пакету (MTU). Дослідження показало, що збільшення розміру пакету позитивно впливає на швидкість передачі в обох режимах. Це пов'язано зі зменшенням кількості заголовків та операцій обробки. Однак, занадто великий розмір пакету зменшує ефективну ємність буфера маршрутизатора (в кількості пакетів), що вимагає балансу.

Зробимо підсумок: розроблена система довела працездатність модифікованого алгоритму Дейкстри. Було показано, що дейтаграмний режим із балансуванням навантаження є більш стійким та ефективним для масивної передачі даних у розподілених мережах, тоді як віртуальний канал краще підходить для мінімізації затримок при менших обсягах даних.

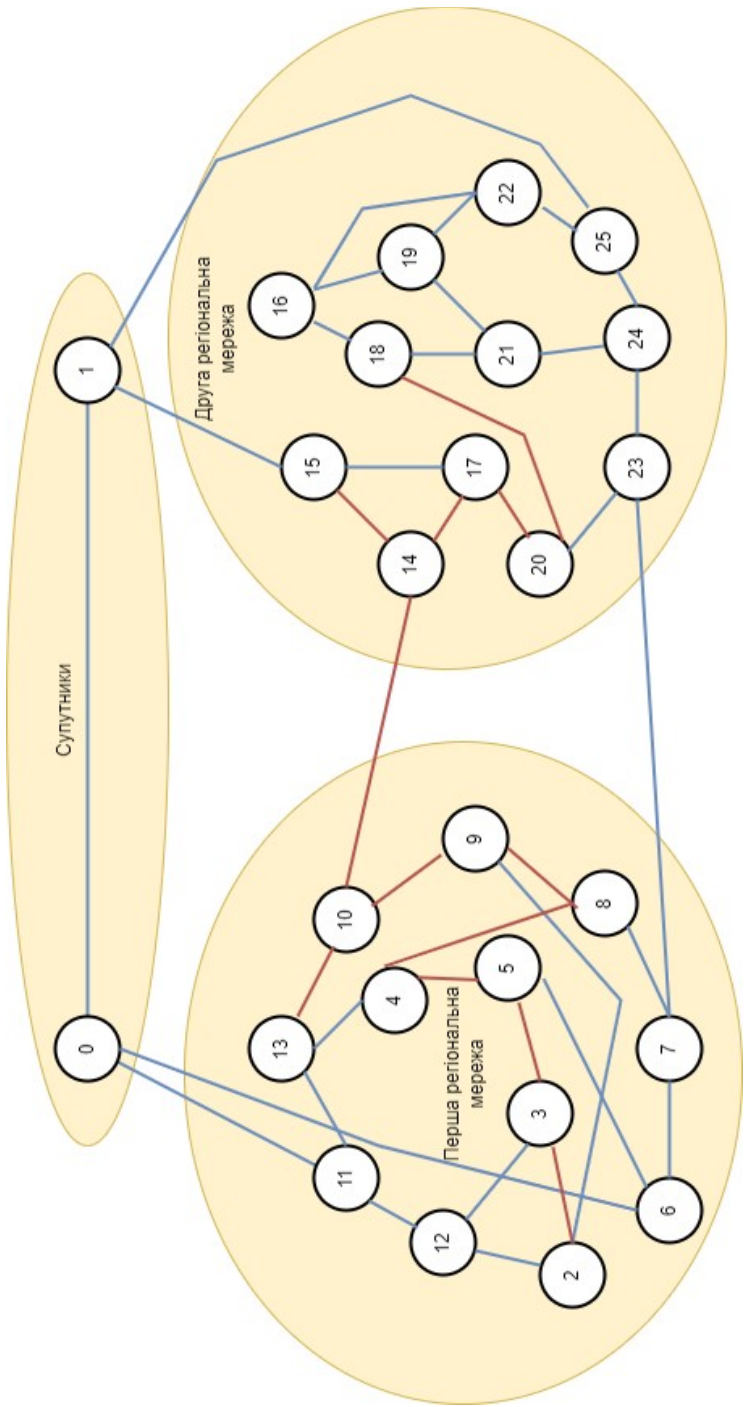
					ІАЛЦ. 467200.004 ПЗ	Аркуш 33
Зм.	Арк	№ докум	Підпис	Дата		

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Комп'ютерні мережі: підручник / Азаров О.Д., Захарченко С.М., Кадук О.В., Орлова М.М., Тарасенко В.П. – Вінниця: ВНТУ. – 2020. – 378 с.
2. Комп'ютерні мережі: навчальний посібник / Азаров О.Д., Захарченко С.М., Кадук О.В., Орлова М.М., Тарасенко В.П. – Вінниця: ВНТУ. – 2013. – 371 с.
3. Є.Буров. Комп'ютерні мережі. Львів. – 2003. – 584 с.
4. Кулаков Ю.О., Жуков І.А. Комп'ютерні мережі: навчальний посібник. – Київ: НАУ. – 2009. – 392 с.
5. Комп'ютерні мережі 1. Основні принципи побудови комп'ютерних мереж. Лабораторні роботи з дисципліни «Комп'ютерні мережі» [Електронний ресурс] : навч. посіб. Для студ. спеціальності 123 «Комп'ютерна інженерія», освітньої програми «Системне програмування та спеціалізовані комп'ютерні системи» / КПІ ім. Ігоря Сікорського ; уклад.: М. М. Орлова, А. А. Крайносвіт, П. А. Сергієнко. – Електронні текстові дані (1 файл: 6,85 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2021. – 165 с

					ІАЛЦ. 467200.004 ПЗ	Аркуш 34
Зм.	Арк	№ докум	Підпи с	Дата		

ДОДАТОК А. КОНФІГУРАЦІЯ МЕРЕЖІ



Зм.	Арк	№ докум	Підпис	Дата